OPLOGS-05: Querying & Parsing Logs

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 5 of 8 | **Created:** December 2025 | **Last Updated:** 10/23/2026

DQL Fundamentals and DPL Pattern Matching

This notebook covers DQL query syntax, filtering, string matching, and DPL (Dynatrace Pattern Language) for extracting structured data from logs.

Table of Contents

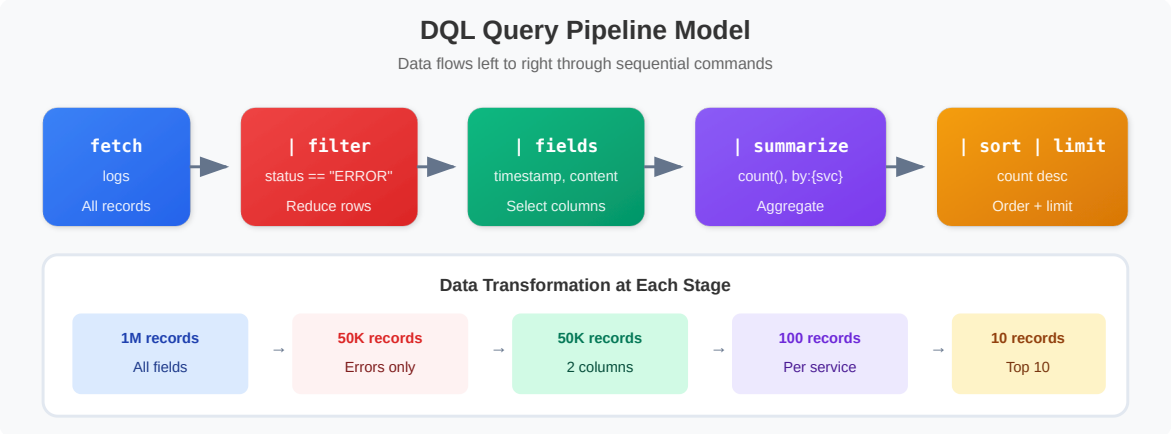
1. DQL Fundamentals
2. Filtering Logs
3. String Matching Functions
4. Field Selection and Transformation
5. DPL (Dynatrace Pattern Language) Parsing
6. Working with JSON Logs
7. Null Handling
8. Advanced Parsing Examples
9. 📝 Summary
10. ➡️ Next Steps
11. 📚 References

Prerequisites

- ✅ Access to a Dynatrace environment with log data
- ✅ Completed OPLOGS-01 through OPLOGS-04
- ✅ Basic understanding of regular expressions (helpful)

1. DQL Fundamentals

Query Structure




```
fetch logs, from: now() - 1h          // Data source and time range
| filter &lt;condition&gt;                    // Filter records
| fields &lt;field1&gt;, &lt;field2&gt;          // Select fields
| sort &lt;field&gt; desc                   // Order results
| limit 100                           // Limit output
```

🚨 Critical DQL Rules (DQL ≠ SQL)

✅ DQL Syntax	❌ SQL Syntax
`filter status == "ERROR"`	`WHERE status = 'ERROR'`
`in(field, {"a", "b"})`	`field IN ('a', 'b')`
`summarize {count = count()}`	`SELECT COUNT(*)`
`isNull(field)`	`field IS NULL`

```
// Basic log query - explore recent logs
fetch logs, from: now() - 1h
| fields timestamp, content, loglevel, dt.entity.host
| limit 10
```

```
// Query with time range options
fetch logs, from: now() - 24h, to: now() - 12h
| summarize {count = count()}, by: {loglevel}
| sort count desc
```

2. Filtering Logs

Comparison Operators

Operator	Description	Example
`==`	Equals	`loglevel == "ERROR"`
`!=`	Not equals	`status != "NONE"`
`>`, `>=`	Greater than	`count > 100`
`<`, `<=`	Less than	`duration < 1s`

Logical Operators

Operator	Description	Example
`AND`	Both conditions	`loglevel == "ERROR" AND isNotNull(host)`
`OR`	Either condition	`loglevel == "ERROR" OR loglevel == "WARN"`
`NOT`	Negate	`NOT contains(content, "health")`

> Compare durations with duration literals — `5s`, `100ms` — never a bare number: `duration < 1000` compares an integer with a duration and silently matches nothing.

```
// Filter by log level
fetch logs, from: now() - 1h
| filter loglevel == "ERROR"
| fields timestamp, content, dt.entity.host
| limit 20
```

```
// Multiple conditions with AND/OR
fetch logs, from: now() - 1h
| filter (loglevel == "ERROR" OR loglevel == "WARN")
        AND isNotNull(dt.entity.host)
| summarize {count = count()}, by: {loglevel, dt.entity.host}
| sort count desc
| limit 15
```

```
// Filter using in() for multiple values
fetch logs, from: now() - 1h
| filter in(loglevel, {"ERROR", "WARN", "INFO"})
| summarize {count = count()}, by: {loglevel}
| sort count desc
```

3. String Matching Functions

Function	Description	Example
`contains(str, substr)`	Substring match	`contains(content, "error")`
`startsWith(str, prefix)`	Prefix match	`startsWith(content, "[ERROR]")`
`endsWith(str, suffix)`	Suffix match	`endsWith(content, "failed")`
`matchesPhrase(str, phrase)`	Word boundary match	`matchesPhrase(content, "connection refused")`
`matchesValue(str, pattern)`	Exact or wildcard match	`matchesValue(host, "web-*")`

What each match costs

None of these functions prunes buckets the way `bucket:{...}` does: every one read all 3.51M records in scope in the test below. What differs is how much of each record they

have to read. Dynatrace's guidance is to use `==` for known values and token matching (`matchesPhrase`, `~`) rather than substring matching on text. Measurements on one validation tenant point the same way (10/23/2026, a 15-minute window that scanned 731.8 MB unfiltered, `summarize count()` after each filter):

Filter	Scanned	Rows
`k8s.namespace.name == "easytrade"`	419.4 MB	83,324
`matchesValue(k8s.namespace.name, "easytrade")`	419.4 MB	83,324
`contains(k8s.namespace.name, "easytrade")`	488.3 MB	83,324
`matchesValue(k8s.namespace.name, "easy*")`	502.9 MB	84,929
`contains(content, "error")`	371.1 MB	15,652
`matchesValue(content, "*error*")`	371.1 MB	18,811
`matchesPhrase(content, "error")`	341.4 MB	18,669
`status == "ERROR"`	243.2 MB	29,637

Three patterns held on that tenant:

- **Structured field:** an exact match (`==`, or `matchesValue` without a wildcard) scanned 14–17% less than `contains` or a wildcard.
- **Free text:** `contains` and `matchesValue` scanned the same bytes and differed only in which rows matched. The default `matchesValue` ignores case, so it found 18,811 rows to `contains`' 15,652.
- **Error records:** filtering the parsed `status` field beat every text search for "error". It scanned the least and returned the most error records.

In community practice the size of these gaps depends on how your data is laid out, so read the table as direction, not as a rate. Compare your own queries with the `scannedBytes` value in the query metadata.

> **Sources:** Anomaly Detection DQL optimization guide (DT docs) — *"Use the == operator to filter known values. This is the most efficient way to narrow your alerting scope."*; on `contains` versus `matchesPhrase`, *"it is less efficient because token matching is not applied."*

```
// Search for specific content patterns
fetch logs, from: now() - 1h
| filter contains(content, "Exception")
| fieldsAdd content_preview = substring(content, from: 0, to: 100)
| fields timestamp, content_preview, dt.entity.host
| limit 15
```

```
// Use matchesPhrase for word-boundary matching
fetch logs, from: now() - 1h
| filter matchesPhrase(content, "connection")
| fieldsAdd content_preview = substring(content, from: 0, to: 120)
| summarize {count = count()}, by: {content_preview}
| sort count desc
| limit 10
```

```
// Exclude patterns with NOT
// status == "ERROR" covers every error-class level (ERROR, SEVERE, CRITICAL,
FATAL, …);
// loglevel == "ERROR" alone misses SEVERE and the rest
fetch logs, from: now() - 1h
| filter status == "ERROR"
| filter NOT matchesPhrase(content, "health")
| filter NOT matchesPhrase(content, "heartbeat")
| fieldsAdd content_preview = substring(content, from: 0, to: 100)
| summarize {count = count()}, by: {content_preview}
| sort count desc
| limit 10
```

4. Field Selection and Transformation

Field Commands

Command	Description
`fields`	Select specific fields only
`fieldsAdd`	Add computed fields
`fieldsRemove`	Remove fields

```
// Add computed fields
fetch logs, from: now() - 1h
| filter status == "ERROR"
| fieldsAdd severity = if(contains(content, "critical"), "CRITICAL",
                       else: if(contains(content, "fatal"), "FATAL",
                       else: loglevel))
| fieldsAdd content_length = stringLength(content)
| fields timestamp, severity, content_length, dt.entity.host
| limit 20
```

```
// String manipulation functions
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| fieldsAdd pod_short = substring(k8s.pod.name, from: 0, to: 30)
| fields timestamp, k8s.namespace.name, pod_short, loglevel
| limit 10
```

5. DPL (Dynatrace Pattern Language) Parsing

DPL extracts structured data from unstructured log content.

DPL Matchers

Matcher	Description	Matches
`INT`	Integer	`42`, `-17`
`DOUBLE`	Decimal	`3.14`, `-0.5`
`IPADDR`	IP address	`192.168.1.1`, `::1`
`LD`	Line data (to delimiter)	Any text
`WORD`	Word characters	`hello`, `user123`
`SPACE`	Whitespace	spaces, tabs
`JSON`	JSON structure	`{"key": "value"}`

Pattern Syntax

Syntax	Description
`MATCHER:fieldname`	Extract to named field
`MATCHER`	Match but don't extract
`MATCHER?`	Optional matcher
`'literal'`	Match exact string
`(opt1|opt2)`	Match alternatives

A `parse` pattern must match from the **start** of the string (trailing text is fine). To find a value anywhere, prefix `DATA?` before a literal, or use `parseAll()` for matcher-only patterns — see FAQ-15 §5.

```
// Parse log level from content (e.g., "[ERROR] message")
fetch logs, from: now() - 1h
| filter startsWith(content, "[")
| parse content, "'[' LD:parsed_level ']'"
// keep only supported loglevel values — bracketed timestamps and message
fragments are not levels
| fieldsAdd parsed_level = upper(parsed_level)
| filter in(parsed_level, {"EMERGENCY", "ALERT", "CRITICAL", "SEVERE",
"ERROR", "FATAL", "WARN", "NOTICE", "INFO", "DEBUG", "TRACE"})
| summarize {count = count()}, by: {parsed_level}
| sort count desc
| limit 10
```

```
// Parse HTTP-style logs
// Example: GET /api/users 200 45ms -&gt; GET, /api/users, 200
// Returns rows only for single-spaced lines of this exact shape; the range
filter drops counters in the status position
fetch logs, from: now() - 1h
| filter contains(content, "GET") OR contains(content, "POST")
| parse content, "[A-Z]+:method ' ' NSPACE:path ' ' INT:status_code ' '"
| filter status_code &gt;= 100 and status_code &lt;= 599
| summarize {count = count()}, by: {method, status_code}
| sort count desc
| limit 15
```

```
// Parse key-value pairs
// Example: "user=john action=login status=success"
// parseAll returns every user= value in the line (parse would return only
one);
// the !&amp;lt;&amp;lt; lookbehind stops "superuser=root" or "service_user=x"
matching as user=
fetch logs, from: now() - 1h
| fieldsAdd usernames = parseAll(content, "!&amp;lt;&amp;lt;[A-Za-z0-9_] 'user=' [A-
Za-z0-9._@-]+:username")
| filter arraySize(usernames) &amp;gt; 0
| expand username = usernames
| summarize {count = count()}, by: {username}
| sort count desc
| limit 10
```

```
// Parse IP addresses from logs
fetch logs, from: now() - 1h
// parseAll finds every IP anywhere in the line; parse would only match one at
the very start
// IPADDR also matches IPv4-shaped version strings (rv:1.9.2.20) and IPv6
fragments (::0000);
// OPLOGS-08 §3 shows a bounded IPv4 pattern for classification
| fieldsAdd client_ip = parseAll(content, "IPADDR:ip")
| filter arraySize(client_ip) &amp;gt; 0
| expand client_ip
| summarize {count = count()}, by: {client_ip}
| sort count desc
| limit 15
```

6. Working with JSON Logs

Many applications emit JSON-formatted logs. DQL provides tools to work with JSON content.

```
// Find JSON-formatted logs
fetch logs, from: now() - 1h
| filter startsWith(content, "{")
| fieldsAdd content_preview = substring(content, from: 0, to: 150)
| summarize {count = count()}, by: {content_preview}
| sort count desc
| limit 10
```

```
// Parse JSON and extract fields
fetch logs, from: now() - 1h
| filter startsWith(content, "{")
| parse content, "JSON:json_data"
| filter isNotNull(json_data)
| fields timestamp, json_data
| limit 10
```

```
// Access nested JSON fields
fetch logs, from: now() - 1h
| filter startsWith(content, "{")
| parse content, "JSON:json_data"
| filter isNotNull(json_data)
| fieldsAdd error_type = json_data[errorType]
| fieldsAdd status_val = json_data[status]
| filter isNotNull(status_val)
| summarize {count = count()}, by: {status_val, error_type}
| sort count desc
| limit 15
```

7. Null Handling

DQL uses three-valued logic. `NULL` comparisons require special functions.

❌ Wrong	✅ Correct
`field == null`	`isNull(field)`
`field != null`	`isNotNull(field)`

```
// Check for null values
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    with_host = countIf(isNotNull(dt.entity.host)),
    without_host = countIf(isNull(dt.entity.host)),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    without_namespace = countIf(isNull(k8s.namespace.name))
  }
```

```
// Use coalesce for default values
// coalesce returns the first non-null argument: the workload name, else the namespace, else "UNKNOWN".
// (coalesce(loglevel, …) would never fall back: loglevel is never null, and missing levels arrive as "NONE".)
fetch logs, from: now() - 1h
| fieldsAdd owner = coalesce(k8s.workload.name, k8s.namespace.name, "UNKNOWN")
| summarize {count = count()}, by: {owner}
| sort count desc
| limit 15
```

8. Advanced Parsing Examples

```
// Parse exception patterns
// Split the record into whitespace-separated tokens (splitByPattern with SPACE also splits on line breaks) and keep the ones that end in "Exception".
// A pattern like "LD:exception_type 'Exception'" anchors at the start of the line, so it
// captures the whole prefix ("2026-10-02 ERROR c.e.Svc - java.lang.NullPointer") instead
// of the class name; DPL does not backtrack, so a greedy word matcher cannot stop before
// the literal "Exception" either.
fetch logs, from: now() - 24h
| filter contains(content, "Exception")
| fieldsAdd token = splitByPattern(content, "SPACE")
| expand token
| filter endsWith(token, "Exception") OR endsWith(token, "Exception:")
| fieldsAdd exception_type = replaceString(token, ":", "")
| summarize {count = count()}, by: {exception_type}
| sort count desc
| limit 15
```

```
// Parse with optional fields
// Matches "error code=123" or "... error code=123 message=failed" anywhere in
the line
// (DATA? lets the pattern start mid-line; parse otherwise anchors at the
start)
fetch logs, from: now() - 1h
| filter contains(content, "error")
| parse content, "DATA? 'error' SPACE 'code=' INT:error_code (SPACE 'message='
LD:error_msg)?"
| filter isNotNull(error_code)
| summarize {count = count()}, by: {error_code}
| sort count desc
| limit 10
```

```
// Parse alternative formats
// Matches "user=john" or "username=john" anywhere in the line, every
occurrence (parseAll);
// the character class stops at the first character outside it, so
"user=jdoe," yields "jdoe"
// (a trailing LD would take the rest of the line), and the !&lt;&lt;
lookbehind skips "superuser=root"
fetch logs, from: now() - 1h
| fieldsAdd user_values = parseAll(content, "!&lt;&lt;[A-Za-z0-9_]
('user='|'username=') [A-Za-z0-9._@-]+:user_value")
| filter arraySize(user_values) &gt; 0
| expand user_value = user_values
| summarize {count = count()}, by: {user_value}
| sort count desc
| limit 10
```

📝 Summary

In this notebook, you learned:

✅ **DQL fundamentals** - Query structure and syntax rules
✅ **Filtering** - Comparison operators, logical operators, in()
✅ **String matching** - contains, matchesPhrase, startsWith
✅ **Field manipulation** - fieldsAdd, computed fields
✅ **DPL parsing** - Matchers (INT, LD, IPADDR, JSON)
✅ **JSON handling** - Parsing and accessing nested fields
✅ **Null handling** - isNull, isNotNull, coalesce

➡️ Next Steps

Continue to **OPLOGS-06: Topology & Entity Context** to learn about entity context and relationships.

📚 References

- Dynatrace Query Language (DT docs)
- DQL functions (DT docs)
- Dynatrace Pattern Language (DT docs)
- DPL Architect (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.